The Boundary of Nelson's Feasibility Program: Per-Program Proofs Are Feasible, Self-Reference Is Not

Abstract

We formalize in Agda a tree-based equational system combining Guard's syntax (binary trees with functors) and Rose's purely equational proof style [9] (no propositional connectives — only equations and tree induction). The formalization contains: (1) a complete diagonal Gödel II proof [4] (168 lines, 0 postulates); (2) a micro-Nelson instance [7] verifying 56 programs against a fixed target; (3) a uniform proof-construction mechanism (the R/Q technique) producing constant-size, BLevel-0 invariant proofs for all oblivious step functions; (4) the Kritchman–Raz contradiction theorem [6], fully proved. The main conclusion: the *per-program* content of a consistency proof is feasible ($O(1)$, ground-verifiable), but combining these proofs into a system-level consistency statement requires self-reference, which is irreducibly infeasible.

1 The formal system

The system combines two ideas: *Guard*'s syntax of binary trees with typed functors, and *Rose*'s Goodstein-style proof theory [9] where the only judgement form is an equation $t = u$ between terms. There are no propositional connectives, no sequents, no logical rules — only equations and a tree-induction principle (**Schema F**).

Trees

The ground data are finite binary trees:

$$a, b \ ::= \ O \ \mid \ \langle a, b \rangle$$

where O is the leaf and $\langle a, b \rangle$ is the node. We write $\overline{n}$ for the unary encoding of $n \in \mathbb{N}$: $\overline{0} = O$, $\overline{n+1} = \langle O, \overline{n} \rangle$.

Terms and functors

Terms are built from trees, variables, and functor applications. We use $t, u, v, \ldots$ for terms and x, y for variables. There are two sorts of functors: unary ($f, g : \mathrm{Fun}_1$) and binary ($h : \mathrm{Fun}_2$).

f, g	$::=$	I	identity
	$\mid$	π_1	first projection
	$\mid$	π_2	second projection
	$\mid$	$f \circ g$	composition
	$\mid$	$h[f, g]$	binary composition: $t \mapsto h(f(t), g(t))$
	$\mid$	$\mathrm{R}(z, s)$	tree recursion with base z and step s
	$\mid$	K_c	constant c
h	$::=$	P	pairing: $(t, u) \mapsto \langle t, u \rangle$
	$\mid$	C	left projection: $(t, u) \mapsto t$
	$\mid$	$\mathrm{Lf}(f)$	lift: $(t, u) \mapsto f(t)$
	$\mid$	$\mathrm{Ps}(f, h)$	post-compose: $(t, u) \mapsto f(h(t, u))$
	$\mid$	$\mathrm{Fn}(h_1, h_2, h)$	fan: $(t, u) \mapsto h(h_1(t, u),\ h_2(t, u))$
	$\mid$	If	conditional
	$\mid$	Eq	tree equality test

Terms:

$$t, u \ ::= \ O \ \mid \ x \ \mid \ f(t) \ \mid \ h(t, u)$$

An *equation* is a pair $t = u$ of terms.

Embedding trees into terms

Every tree a determines a *closed term* $\hat{a}$:

$$\hat{O} = O, \qquad \widehat{\langle a, b \rangle} = \langle \hat{a}, \hat{b} \rangle.$$

(In the Agda code this function is called `reify`.) Conversely, every closed term reduces to $\hat{a}$ for a unique tree a. The embedding $a \mapsto \hat{a}$ lets us use trees as term-level constants.

Derivation rules

A derivation is a well-founded tree built from the rules below. Every rule may use a hypothesis equation H; we write $H \vdash t = u$ for "$t = u$ is derivable from H."

Computation axioms.

Identity:	$\mathrm{I}(t) = t$
First projection:	$\pi_1(\langle t, u \rangle) = t$
Second projection:	$\pi_2(\langle t, u \rangle) = u$
Constant:	$\mathrm{C}(t, u) = t$
Composition:	$(f \circ g)(t) = f(g(t))$
Binary composition:	$h[f, g](t) = h(f(t),\ g(t))$
Lift:	$\mathrm{Lf}(f)(t, u) = f(t)$
Post-compose:	$\mathrm{Ps}(f, h)(t, u) = f(h(t, u))$
Fan:	$\mathrm{Fn}(h_1, h_2, h)(t, u) = h(h_1(t, u),\ h_2(t, u))$
Constant functor:	$\mathrm{K}_c(t) = c$

Tree recursion.

$$\begin{aligned}
&\text{Base:} && \mathrm{R}(z,s)(O) = z \\
&\text{Step:} && \mathrm{R}(z,s)(\langle t,u\rangle) = s\big(\langle t,u\rangle,\ \langle \mathrm{R}(z,s)(t), \mathrm{R}(z,s)(u)\rangle\big)
\end{aligned}$$

Conditional.

$$\begin{aligned}
&\text{Leaf branch:} && \mathrm{If}(O,\ \langle t,u\rangle) = t \\
&\text{Node branch:} && \mathrm{If}(\langle t_1,t_2\rangle,\ \langle u_1,u_2\rangle) = u_2
\end{aligned}$$

Tree equality.

$$\begin{aligned}
&\mathrm{Eq}(O,O) && = O \\
&\mathrm{Eq}(O,\langle t,u\rangle) && = \langle O,O\rangle \\
&\mathrm{Eq}(\langle t,u\rangle,O) && = \langle O,O\rangle \\
&\mathrm{Eq}(\langle t_1,t_2\rangle,\langle u_1,u_2\rangle) && = \mathrm{If}\big(\mathrm{Eq}(t_1,u_1),\ \langle \mathrm{Eq}(t_2,u_2), \langle O,O\rangle\rangle\big)
\end{aligned}$$

Here Eq returns O for equal trees and $\langle O,O\rangle$ for unequal.

Structural rules.

$$\frac{}{t=t}\ (\mathrm{refl}) \qquad \frac{t=u}{u=t}\ (\mathrm{sym}) \qquad \frac{t=u \qquad u=v}{t=v}\ (\mathrm{trans})$$

$$\frac{t=u}{f(t)=f(u)}\ (\mathrm{cong}_1) \qquad \frac{t=u}{h(t,v)=h(u,v)}\ (\mathrm{cong}_L) \qquad \frac{t=u}{h(v,t)=h(v,u)}\ (\mathrm{cong}_R)$$

Substitution.

$$\frac{H \vdash t=u}{H \vdash t[c/x]=u[c/x]}\ (\mathrm{inst})$$

Hypothesis.

$$\frac{}{H \vdash H}\ (\mathrm{hyp})$$

Schema F (tree induction). If f and g are unary functors, z a term, s a binary functor, and we have derivations of:

$$\begin{aligned}
&f(O) = z, && f(\langle x,y\rangle) = s(\langle x,y\rangle,\ \langle f(x), f(y)\rangle), \\
&g(O) = z, && g(\langle x,y\rangle) = s(\langle x,y\rangle,\ \langle g(x), g(y)\rangle),
\end{aligned}$$

then $f(x) = g(x)$.

Schema F expresses the uniqueness of tree recursion: if f and g satisfy the same recurrence equations (same base z, same step s), they agree on all inputs. Restricted to the unary trees $\overline{n}$, this gives ordinary mathematical induction.

Consistency

The system is *consistent* if there is no derivation of $O = \langle O,O\rangle$. Since O and $\langle O,O\rangle$ are distinct closed terms (leaf vs. node), consistency means the system does not confuse structurally different trees.

2 Proof encoding and ground evaluation

Every derivation rule is assigned a *tag* (a natural number 0–25). A tree a is interpreted as a proof encoding: the function thFun : Tree $\to$ Tree reads the tag from a and reconstructs the equation that a purports to prove, producing $\langle \ulcorner t \urcorner, \ulcorner u \urcorner \rangle$ where $\ulcorner t \urcorner$ is the tree representation of the term t.

The ground evaluator geval takes a coded term (a tree representing a term built from functor applications) and computes its value by dispatching on functor-code tags (numbered 26–32, disjoint from the proof tags). For *ground* coded terms (containing no variables), geval correctly computes the semantic value. For terms with variables, geval returns a stuck (passthrough) result.

The proof checker $\mathrm{checkProof}_2(a)$ runs thFun to extract the equation encoded by a, then checks whether geval produces the same tree on both sides. The distinction between ground and stuck is exactly the distinction between *oblivious* and *non-oblivious* proofs.

3 Programs and iteration

A *program* is a unary functor of the form $\mathrm{iterate}(f, O) = \mathrm{R}(O, s_f)$ where f is a step function and s_f extracts the right recursive result and applies f. It satisfies:

$$\begin{aligned} \mathrm{iterate}(f, O)(O) &= O, \\ \mathrm{iterate}(f, O)(\langle t, u \rangle) &= f\big(\mathrm{iterate}(f, O)(u)\big). \end{aligned}$$

Applied to $\overline{n}$, this gives $f^n(O)$. The *code size* of a program p is $|p|$, the number of nodes in the tree encoding of p's functor code.

4 The R/Q technique

Two lemmas, both proved by **Schema F** at BLevel 0 (no variable instantiation):

Lemma 1 (R — collapse)**.** *For all terms t:*

$$\mathrm{If}(t,\ \langle \langle O, O \rangle, \langle O, O \rangle \rangle) \ = \ \langle O, O \rangle.$$

Proof. Define $R(t) = \mathrm{If}(t, \langle \langle O, O \rangle, \langle O, O \rangle \rangle)$. Base: $R(O) = \langle O, O \rangle$ (leaf branch of If). Step: $R(\langle x, y \rangle) = \langle O, O \rangle$ (node branch). Both match $\mathrm{K}_{\langle O,O \rangle}$. **Schema F** gives $R = \mathrm{K}_{\langle O,O \rangle}$. □

Lemma 2 (Q — for $a \neq b$ with one component O)**.** *Let $a = \langle O, O \rangle$, $b = O$. For all terms t:*

$$\mathrm{If}\big(\mathrm{Eq}(t, a),\ \langle \mathrm{Eq}(t, b), \langle O, O \rangle \rangle\big) \ = \ \langle O, O \rangle.$$

Proof. Define $Q(t) = \mathrm{If}(\mathrm{Eq}(t, \langle O, O \rangle),\ \langle \mathrm{Eq}(t, O), \langle O, O \rangle \rangle)$. Base: $Q(O)$. Since $\mathrm{Eq}(O, \langle O, O \rangle) = \langle O, O \rangle$, If takes the node branch, giving $\langle O, O \rangle$. Step: $Q(\langle x, y \rangle)$. Since $\mathrm{Eq}(\langle x, y \rangle, O) = \langle O, O \rangle$, the pair argument becomes $\langle \langle O, O \rangle, \langle O, O \rangle \rangle$. By R, the result is $\langle O, O \rangle$. **Schema F** closes: $Q = \mathrm{K}_{\langle O,O \rangle}$. □

The key: Q captures a case analysis without case-splitting. When $t = a$, the first comparison returns O and If falls through to the second comparison, which detects $t \neq b$ (since $a \neq b$). When $t \neq a$, the first comparison returns $\langle O, O \rangle$ and If short-circuits. Both branches produce $\langle O, O \rangle$, so **Schema F** applies.

5 Three proof families

For step functions $f = \mathrm{P}[g, h]$ where $g, h \in \{I, \pi_1, \pi_2, \mathrm{K}_O, \mathrm{K}_{\langle O,O\rangle}\}$, and target $\xi = \langle\langle O, O\rangle, O\rangle$, three parametric proof constructors handle all oblivious cases. In each case, the theorem proved is: for all terms t, $\mathrm{Eq}(\mathrm{iterate}(f, O)(t),\ \hat{\xi}) = \langle O, O\rangle$.

Family A: Constant g, first-component mismatch

If $g = \mathrm{K}_c$ with $c \neq \langle O, O\rangle$ (so $\mathrm{Eq}(c, \langle O, O\rangle) = \langle O, O\rangle$), then in the step case:

$$\mathrm{Eq}\big(\langle c, \ldots\rangle,\ \langle\langle O, O\rangle, O\rangle\big) \ = \ \mathrm{If}\big(\underbrace{\mathrm{Eq}(c, \langle O, O\rangle)}_{=\,\langle O,O\rangle},\ \ldots\big) \ = \ \langle O, O\rangle.$$

The first-component comparison short-circuits. **5 programs** (all with $g = \mathrm{K}_O$).

Family B: Non-constant g, $h = \mathrm{K}_{\langle O,O\rangle}$

The second-component comparison gives $\mathrm{Eq}(\langle O, O\rangle, O) = \langle O, O\rangle$. The If argument becomes $\langle\langle O, O\rangle, \langle O, O\rangle\rangle$. R collapses it. **3 programs** ($g \in \{I, \pi_1, \pi_2\}$).

Family C: $g = h$ (same function), Q lemma

$f(t) = \langle g(t), g(t)\rangle$. The comparison with $\xi = \langle\langle O, O\rangle, O\rangle$ gives:

$$\mathrm{If}\big(\mathrm{Eq}(g(r), \langle O, O\rangle),\ \langle \mathrm{Eq}(g(r), O), \langle O, O\rangle\rangle\big) \ = \ Q(g(r)) \ = \ \langle O, O\rangle,$$

where r is the recursive result. **3 programs** ($g = h \in \{I, \pi_1, \pi_2\}$).

Total: 11 programs with full equational derivations. All 23 $\mathrm{P}[g, h]$ combinations pass $\mathrm{checkProof}_2$ (geval verification).

6 The micro-Nelson instance

`MicroNelson.agda` enumerates 28 step functions (all $\mathrm{P}[g, h]$ with $g, h \in \{I, \pi_1, \pi_2, \mathrm{K}_O, \mathrm{K}_{\langle O,O\rangle}\}$, plus 5 base functors) in two modes (iterate + direct), giving 56 programs. The target is $\xi_4 = \langle\langle\langle O, O\rangle, O\rangle, \langle O, O\rangle\rangle$, chosen deeper than any $f(O)$ for depth-1 step functions.

All 56 satisfy $\mathrm{checkProof}_2(\mathrm{mkTest}(p, \xi_4)) = \mathrm{true}$, verified by Agda `refl`. The adversarial program $\mathrm{K}_{\hat{\xi}_4}$ (which always outputs ξ_4) correctly returns `false`.

This is a complete pigeonhole instance at constructor-depth ≤ 1: every program of bounded depth is accounted for, and ξ_4 is not in any program's range.

7 The Kritchman–Raz framework

The Chaitin machine [2] C_l enumerates trees and checks each one as a potential proof encoding. If the system proves its own consistency, the search terminates and C_l outputs a string ξ with $K(\xi) > l$ [5]. But $|C_l|$ is bounded: the step function is fixed and l enters only as data via binary encoding, contributing $O(\log l)$ to the code size. For large l, $|C_l| < l$, so C_l itself witnesses $K(\xi) \leq |C_l| < l$, contradicting $K(\xi) > l$.

Theorem 3 (`krClean`, fully proved in Agda)**.** *Given:*

1. *a Chaitin machine family* $cm : \mathbb{N} \to \mathrm{Fun}_1$,
2. *a threshold* T *with* $|cm(l)| \leq l$ *for* $l \geq T$,
3. *for each* $l \geq T$*: a tree* ξ*, a clock* c*, a proof that* $cm(l)$ *outputs* ξ *at clock* c*, and a proof that no* Fun_1 *of size* $\leq l$ *outputs* ξ*,*

then $\perp$*.*

The proof is direct: $cm(T)$ outputs ξ at clock c and has $|cm(T)| \leq T$, contradicting the incompressibility of ξ.

8 The boundary

The formalization reveals a sharp decomposition:

Component	**Status**	**Obstacle**
Per-program invariant	Feasible	None ($O(1)$, BLevel 0)
Ground verification	Feasible	None ($\mathrm{checkProof}_2$)
Pigeonhole (finite l)	Feasible	None (enumeration)
$\forall p.\ \lvert p\rvert \leq l \Rightarrow \ldots$	Infeasible	Self-reference
Chaitin machine as Fun_1	Infeasible	Self-reference
$\lvert C_l\rvert < l$ for large l	Infeasible	Self-reference

The transition from feasible to infeasible is precisely the transition from "for each specific program p" to "for all programs of size $\leq l$."

- "For each p": the R/Q technique produces a constant-size proof. The proof is *oblivious*: it examines only the structural mismatch between p's output format and ξ, not what p computes.
- "For all p": requires the system to enumerate its own programs, count them (pigeonhole), and recognize that the Chaitin search procedure is itself a program (self-reference). This is the same obstacle as in the diagonal proof, repackaged.

The diagonal proof (`GodelII.agda`) concentrates self-reference in the substitution machinery. The Kritchman–Raz approach distributes it across counting and search. Both are blocked by the same fundamental constraint: the system cannot fully reason about its own proof structure.

9 Implications for Nelson's program

1. **The per-program feasibility claim holds.** Each invariant proof is $O(1)$, BLevel 0, and ground-verifiable. The R/Q technique provides this uniformly for all oblivious programs. The geval evaluator extends coverage to non-oblivious programs at the metalevel. This is the positive content of Nelson's program [7].

2. **The organizational infeasibility persists.** Combining per-program proofs into a system-level consistency statement requires self-reference — whether via diagonalization or via counting. The Kritchman–Raz approach [6], which avoids the diagonal lemma entirely, hits the same wall. The obstacle is not an artifact of the proof method.

3. **The boundary is intrinsic.** The oblivious/non-oblivious distinction is a local version of the each/all boundary. Oblivious proofs work by structure alone (the output format mismatches ξ). Non-oblivious proofs require knowing the actual computed value — a miniature version of the self-reference problem. The BLevel hierarchy is related to Aschieri and Berardi's backtracking levels [1].

4. **Bounded consistency is feasible.** For a fixed depth bound d, the micro-Nelson instance gives a complete, computationally verified consistency proof: every program of depth $\leq d$ is checked, and all pass. This does not extend to unbounded d without self-reference. This connects to the proof-length phenomena studied by Pudlák [8] and the feasible arithmetic of Cook [3].

10 Formalization summary

All results are formalized in Agda 2.9.0 with `--safe --without-K --exact-split`. Zero postulates throughout.

File	Lines	Content
GodelII.agda	168	Diagonal Gödel II
MicroNelson.agda	258	56 programs, $\text{checkProof}_2 = \text{true}$
AutoProof.agda	754	R/Q library, 3 proof families, 11 derivations
Chaitin.agda	395	Binary encoding, tree enum., `krClean`
KRShortProof.agda	310	Case 1 template (first-component mismatch)
DoublerProof.agda	430	Case 3 template (Q lemma)
GeneralQ.agda	443	Cases 2, 4; tripler, mixed
SwapProof.agda	492	Case 4 template (h_2 helper)
Guard/ total	∼40 files	Full system including infrastructure

References

[1] Federico Aschieri and Stefano Berardi. Interactive learning-based realizability for Heyting arithmetic with EM1. *Logical Methods in Computer Science*, 7(2), 2011.

[2] Gregory J. Chaitin. Information-theoretic limitations of formal systems. *Journal of the ACM*, 21(3):403–424, 1974.

[3] Stephen A. Cook. Feasibly constructive proofs and the propositional calculus. *Proceedings of the 7th Annual ACM Symposium on Theory of Computing*, pages 83–97, 1975.

[4] Kurt Gödel. Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I. *Monatshefte für Mathematik und Physik*, 38:173–198, 1931.

[5] Andrei N. Kolmogorov. Three approaches to the quantitative definition of information. *Problems of Information Transmission*, 1(1):1–7, 1965.

[6] Shira Kritchman and Ran Raz. The surprise examination paradox and the second incompleteness theorem. *Notices of the AMS*, 57(11):1454–1458, 2010.

[7] Edward Nelson. Predicative arithmetic. *Mathematical Notes*, 1986.

[8] Pavel Pudlák. The lengths of proofs. In *Handbook of Proof Theory*, pages 547–637. Elsevier, 1998.

[9] Gene F. Rose. Propositional calculus and realizability. In *Transactions of the American Mathematical Society*, volume 75, pages 1–19, 1953.